# CS108 Endsem TSC

—

Sushant, Ansh, Shanmathi

# make

- The make program uses the makefile data base and the last-modification times of the files to decide which of the files need to be updated. For each of those files, it issues the recipes recorded in the data base.
- A simple makefile consists of “rules” with the following shape:

```
target ... : prerequisites ...
        recipe
        ...
        ...
```

- A target is usually the name of a file that is generated by a program. A target can also be the name of an action to carry out, such as ‘clean’ (Phony targets!).
- A prerequisite is a file that is used as input to create the target.
- Arecipe is an action that make carries out. A recipe may have more than one command, either on the same line or each on its own line. **You need to put a tab character at the beginning of every recipe line!** If you prefer to prefix your recipes with a character other than tab, you can set the .RECIPEPREFIX variable to an alternate character

# Simple makefile

- Simple makefile demo
  a. How does make process
  b. Variables
  c. Implicit rules
  d. Grouping targets instead of prereqs
- Makefiles contain five kinds of things: explicit rules, implicit rules, variable definitions, directives, and comments.
- GNU make does its work in two distinct phases.
  a. First, it reads all the makefiles, included makefiles, etc. and internalizes all the variables and their values and implicit and explicit rules, and builds a dependency graph of all the targets and their prerequisites.
  b. Then, make uses this internalized data to determine which targets need to be updated and run the recipes necessary to update them.

# Explicit Rules

- A rule appears in the makefile and says when and how to remake certain files, called the rule's targets. It lists the other files that are the prerequisites of the target, and the recipe to use to create or update the target.
- A rule tells make two things: when the targets are out of date, and how to update them when necessary.
- Order-only prerequisites
- Can use wildcards!
- If a rule has no prerequisites or recipe, and the target of the rule is a nonexistent file, then make imagines this target to have been updated whenever its rule is run. This implies that all targets depending on this one will always have their recipe run.
- Static pattern rules! (targets ...: target-pattern: prereq-patterns ...)

# Phony targets :)

- A phony target is one that is not really the name of a file; rather it is just a name for a recipe to be executed when you make an explicit request. There are two reasons to use a phony target: to avoid a conflict with a file of the same name, and to improve performance.
- .PHONY : clean
  explicitly declares the target clean to be phony, so that ‘make clean’ will run the recipe for the clean target irrespective of whether there is a file named ‘clean’
- Prerequisites of .PHONY are always interpreted as literal target names, never as patterns (even if they contain ‘%’ characters).
- When one phony target is a prerequisite of another, it serves as a subroutine of the other.

# “What is wild and where is the card”

- A single file name can specify many files using wildcard characters. The wildcard characters in make are ‘*’, ‘?’ and ‘[...]’, the same as in the Bourne shell. For example, *.c specifies a list of all the files (in the working directory) whose names end in ‘.c’.
- Wildcard expansion is performed by make automatically in targets and in prerequisites. In recipes, the shell is responsible for wildcard expansion.
- In other contexts, (eg defining a variable) wildcard expansion happens only if you request it explicitly with the wildcard function.
- $(wildcard pattern...)
  This string, used anywhere in a makefile, is replaced by a space-separated list of names of existing files that match one of the given file name patterns.

# Implicit rules

- Implicit rules tell make how to use customary techniques so that you do not have to specify them in detail when you want to use them. For example, there is an implicit rule for C compilation. File names determine which implicit rules are run.  Either write a rule with no recipe, or don't write a rule at all - then make figures out which implicit rule to run.
- Note that explicit prerequisites do not influence implicit rule search. For example, consider this explicit rule:

```
foo.o: foo.p
```

The prerequisite on `foo.p` does not necessarily mean that `make` will remake `foo.o` according to the implicit rule to make an object file, a `.o` file, from a Pascal source file, a `.p` file. For example, if `foo.c` also exists, the implicit rule to make an object file from a C source file is used instead, because it appears before the Pascal rule in the list of predefined implicit rules (see Section 10.2 [Catalogue of Built-In Rules], page 122).

If you do not want an implicit rule to be used for a target that has no recipe, you can give that target an empty recipe by writing a semicolon (see Section 5.9 [Defining Empty Recipes], page 62).

# Variables

It's important to understand this two-phase approach because it has a direct impact on how variable and function expansion happens; this is often a source of some confusion when writing makefiles. Below is a summary of the different constructs that can be found in a makefile, and the phase in which expansion happens for each part of the construct.

We say that expansion is *immediate* if it happens during the first phase: `make` will expand that part of the construct as the makefile is parsed. We say that expansion is *deferred* if it is not immediate. Expansion of a deferred construct part is delayed until the expansion is used: either when it is referenced in an immediate context, or when it is needed during the second phase.

You may not be familiar with some of these constructs yet. You can reference this section as you become familiar with them, in later chapters.

## Variable Assignment

Variable definitions are parsed as follows:

```
immediate = deferred
immediate ?= deferred
immediate := immediate
immediate ::= immediate
immediate :::= immediate-with-escape
immediate += deferred or immediate
immediate != immediate
```

# Some automatic variables

- $@ - file name of target
- $< - name of 1st prerequisite
- $? -  The names of all the prerequisites that are newer than the target, with spaces between them.
- $^ - The names of all the prerequisites, with spaces between them.

# Some useful functions

$(function arguments) or ${function arguments} to call

- $(subst from,to,text) -  each occurrence of from in text is replaced by to.
- $(patsubst pattern,replacement,text) - Finds whitespace-separated words in text that match pattern and replaces them with replacement.
- $(findstring find,in) - Searches in for an occurrence of find. If it occurs, the value is find; otherwise, the value is empty.
- $(filter pattern...,text) - Returns all whitespace-separated words in text that do match any of the pattern words
- $(filter-out pattern...,text) - exact opposite of the filter function
- $(sort list) - Sorts the words of list in lexical order, removing duplicate words.

# Some useful functions

- $(dir names ...) -  Extracts the directory-part of each file name in names. The directory-part of the file name is everything up through (and including) the last slash in it. If the file name contains no slash, directory part is ‘./’
- $(notdir names...) - Opposite of dir
- $(addprefix prefix,names...) - prefix is prepended to the front of each individual name in names.
- $(suffix names...) - Extracts the suffix of each file name in names (everything starting with the last period). basename function does the opposite.
- $(join list1,list2) - concatenate the arguments word by word

# Git - go through the ‘git help <command>’ for all these

- **git help - who to ask for help when you forgot something :) (it’s man pages for git commands)**
- git config - Get and set repository or global options

  git config --global user.name “Me Myself and I”

- git init - Create an empty Git repository or reinitialize an existing one
- git add - Add file contents to the index (staging area)
- git commit - Create a new commit containing the current contents of the index and the given log message describing the changes.
- git log - Show commit logs
- git status - Displays paths that have differences between the index file and the current HEAD commit, paths that have differences between the working tree and the index file, and paths in the working tree that are not tracked by Git (and are not ignored by gitignore)
- git show - Show various types of objects. For commits it shows the log message and textual diff.
- git merge - Join two or more development histories together

```
Assume the following history exists and the current branch is "master":

              A---B---C topic
             /
        D---E---F---G master

Then "git merge topic" will replay the changes made on the topic branch since it diverged from master (i.e.,
E) until its current commit (C) on top of master, and record the result in a new commit along with the names
of the two parent commits and a log message from the user describing the changes. Before the operation,
ORIG_HEAD is set to the tip of the current branch (C).

              A---B---C topic
             /         \
        D---E---F---G---H master
```

# Git

- git stash -  The command saves your local modifications away and reverts the working directory to match the HEAD commit.
- git diff - Show changes between commits, commit and working tree, etc
- git rebase - Reapply commits on top of another base tip
- git checkout - Switch branches or restore working tree files
- git switch - Switch branches
- git branch - List, create, or delete branches
- git reset -  Reset current HEAD to the specified state
- git revert - Given one or more existing commits, revert the changes that the related patches introduce, and record some new commits that record them. This requires your working tree to be clean
- git restore - Restore specified paths in the working tree with some contents from a restore source.
  See "Reset, restore and revert" in git(1) for the differences between the three commands.

# Spreadsheets

- Sheet has rows and columns :)
- Specify a cell with <Column><Row> eg A8, ZA102
- Ranges - A1:J11 selects 110 cells
- ‘=’ starts a formula
- Show formulae via ctrl+`
- Referencing cells in another sheet: Sheet1!A1
- Paste special to paste cells in a certain format (eg paste as values)
- Functions: SUM, AVERAGE, COUNT, IF, COUNTIF, SUMIF
- Sort and filter
- VLOOKUP
- Conditional formatting

# Tips for a nice endsem :)

- Before exam: Practice - Pyqs on bodhitree!
- Before exam: Go through functions and stuff which save time - at least know of their existence and where to refer them in exam. Applies especially for python and bash awk.
- During exam: Stay confident :) you'll be able to do it
- During exam: Documentation exists - use it wisely
- After exam: Congratulations!

# ALL THE BEST!